

Partial Dependence Plots for LightGBM Model

```
import lightgbm as lgb
import pandas as pd
import mlflow
from sklearn.inspection import PartialDependenceDisplay
from smart_goals.predict import load_data
from snowflake.snowpark import Session
import matplotlib.pyplot as plt
```

```
MODEL_NAME = "smart_goals"
TARGET_COL = "SALES_PER_GOAL_HOUR"
DEFAULT_MLFLOW_URI = "https://mlflow.analytics-dev.tapestry.com"
MLFLOW_EXPERIMENT_NAME = "smart_goals_training"
RUN_ID = "40d03b0e9cb44724b376bdfefbadfbd8"

mlflow.end_run()
```

View run likeable-worm-333 at: <https://mlflow.analytics-dev.tapestry.com/#/experiments/81/r>

View experiment at: <https://mlflow.analytics-dev.tapestry.com/#/experiments/81>

```
# Get the MLFlow Run for this model to save these plots along with it
mlflow.set_tracking_uri(DEFAULT_MLFLOW_URI)
mlflow.set_experiment(MLFLOW_EXPERIMENT_NAME)

model = mlflow.sklearn.load_model(f'runs://{RUN_ID}/model')
```

Downloading artifacts: 100%| | 5/5 [00:00<00:00, 18.83it/s]

```
mlflow.start_run(run_id=RUN_ID, experiment_id='81')
```

```
session = Session.builder.getOrCreate()
session.use_database("DEV_DS")
session.use_schema("DBT_DWERNERSEXTON")
data = load_data(session)
```

```
2026-02-19 15:04:54,904 - smart_goals.predict - INFO - Using Snowflake database: DEV_DS, sch
2026-02-19 15:04:54,905 - smart_goals.predict - INFO - Loading data from Snowflake table: ZII
/home/sagemaker-user/smart-goals/src/smart_goals/data.py:96: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

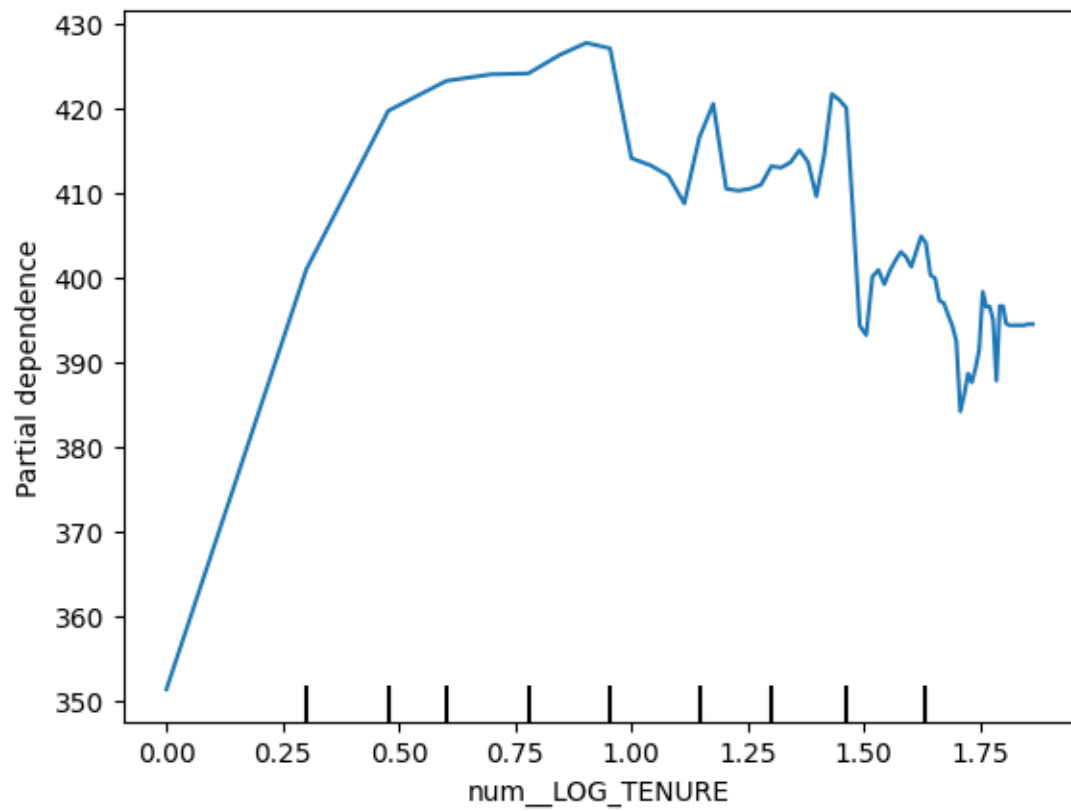
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["BUSINESS_DATE"] = pd.to_datetime(df["BUSINESS_DATE"])
2026-02-19 15:05:07,947 - smart_goals.predict - INFO - Loaded 193833 rows from Snowflake
```

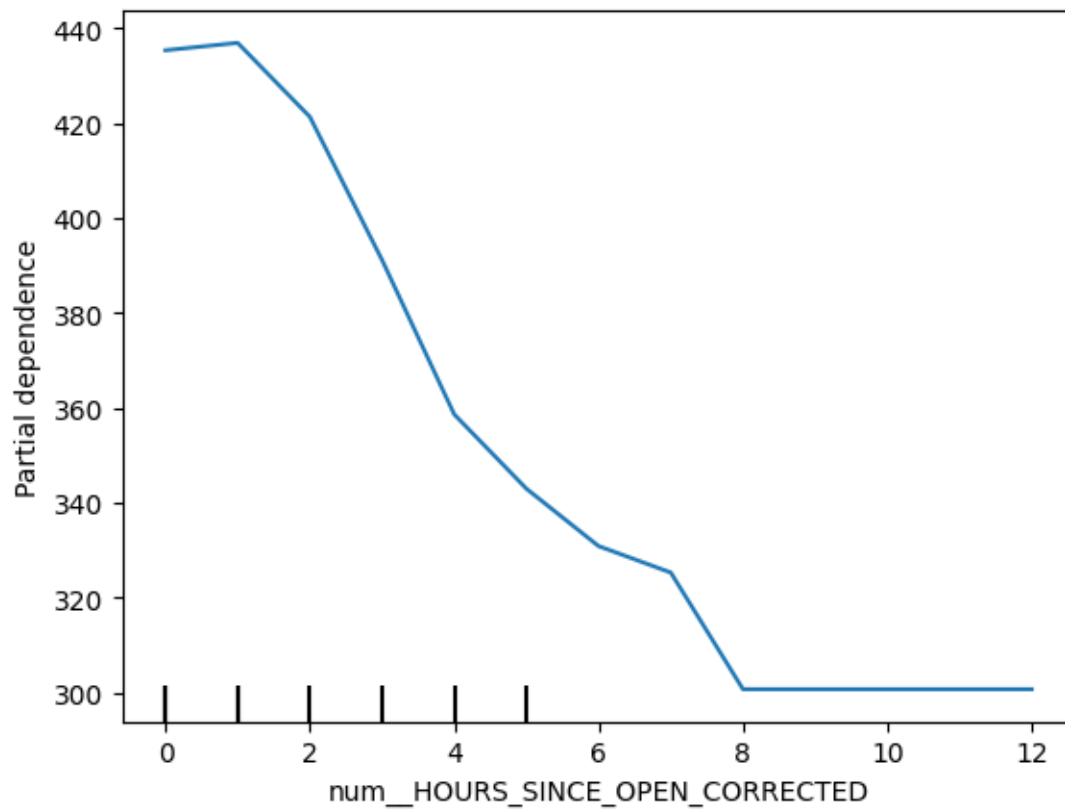
```
X_form = model.named_steps['preprocessing'].transform(data.drop(columns=[TARGET_COL]))
```

```
clf = model.named_steps['regressor']
```

```
# Save the plot to MLFlow
PartialDependenceDisplay.from_estimator(clf, X_form, ['num__LOG_TENURE'])
mlflow.log_figure(plt.gcf(), "partial_dependence_plot.png")
```



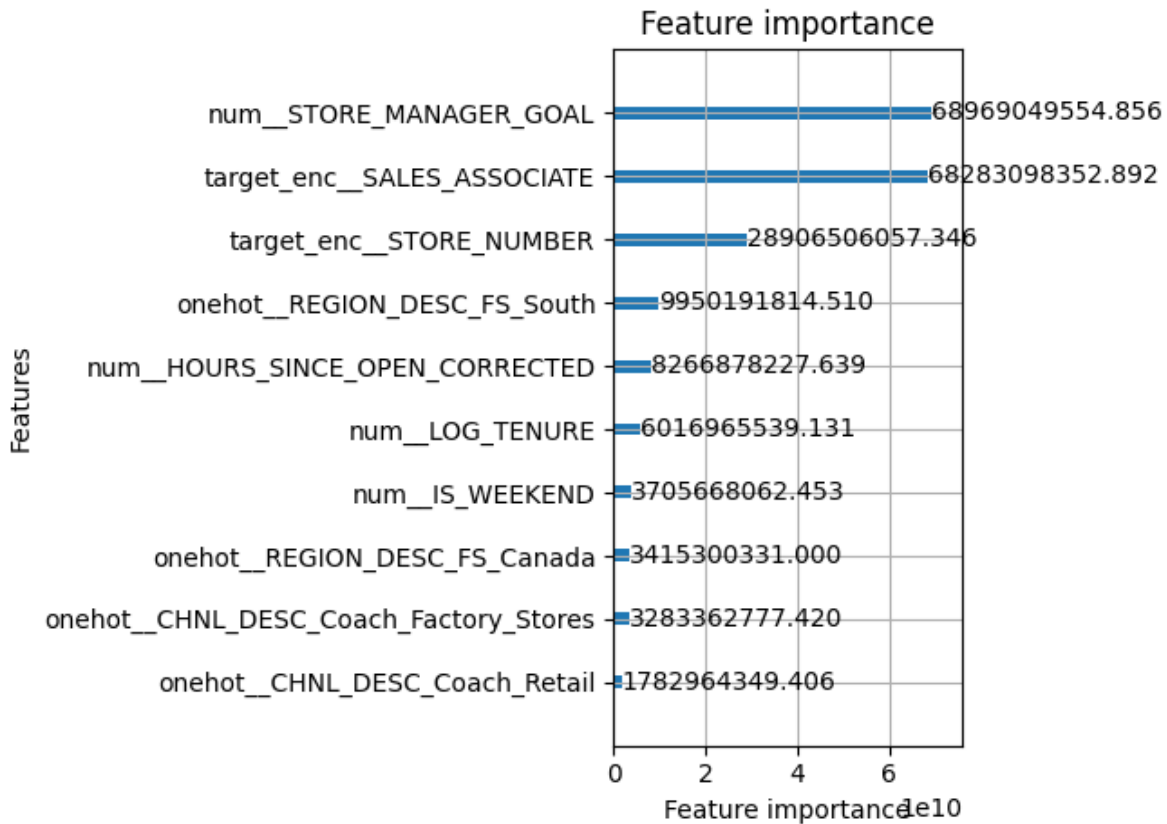
```
PartialDependenceDisplay.from_estimator(clf, X_form, ['num_HOURS_SINCE_OPEN_CORRECTED'])  
mlflow.log_figure(plt.gcf(), "partial_dependence_plot_hours_since_open.png")
```



```
## Feature Importance for LightGBM Model
```

```
plt.figure(figsize=(20, 30))  
lgb.plot_importance(clf, importance_type='gain', max_num_features=10)  
plt.tight_layout(w_pad=1000)  
mlflow.log_figure(plt.gcf(), "feature_importance.png")
```

<Figure size 2000x3000 with 0 Axes>



```
from scipy import stats

# Get predictions and residuals
y_true = data[TARGET_COL]
y_pred = model.predict(data.drop(columns=[TARGET_COL]))
residuals = y_true - y_pred

# Create the 4 diagnostic plots
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# 1. Residuals vs Fitted
axes[0, 0].scatter(y_pred, residuals, alpha=0.5)
axes[0, 0].axhline(y=0, color='r', linestyle='--')
axes[0, 0].set_xlabel('Fitted values')
axes[0, 0].set_ylabel('Residuals')
axes[0, 0].set_title('Residuals vs Fitted')

# 2. Q-Q Plot
```

```

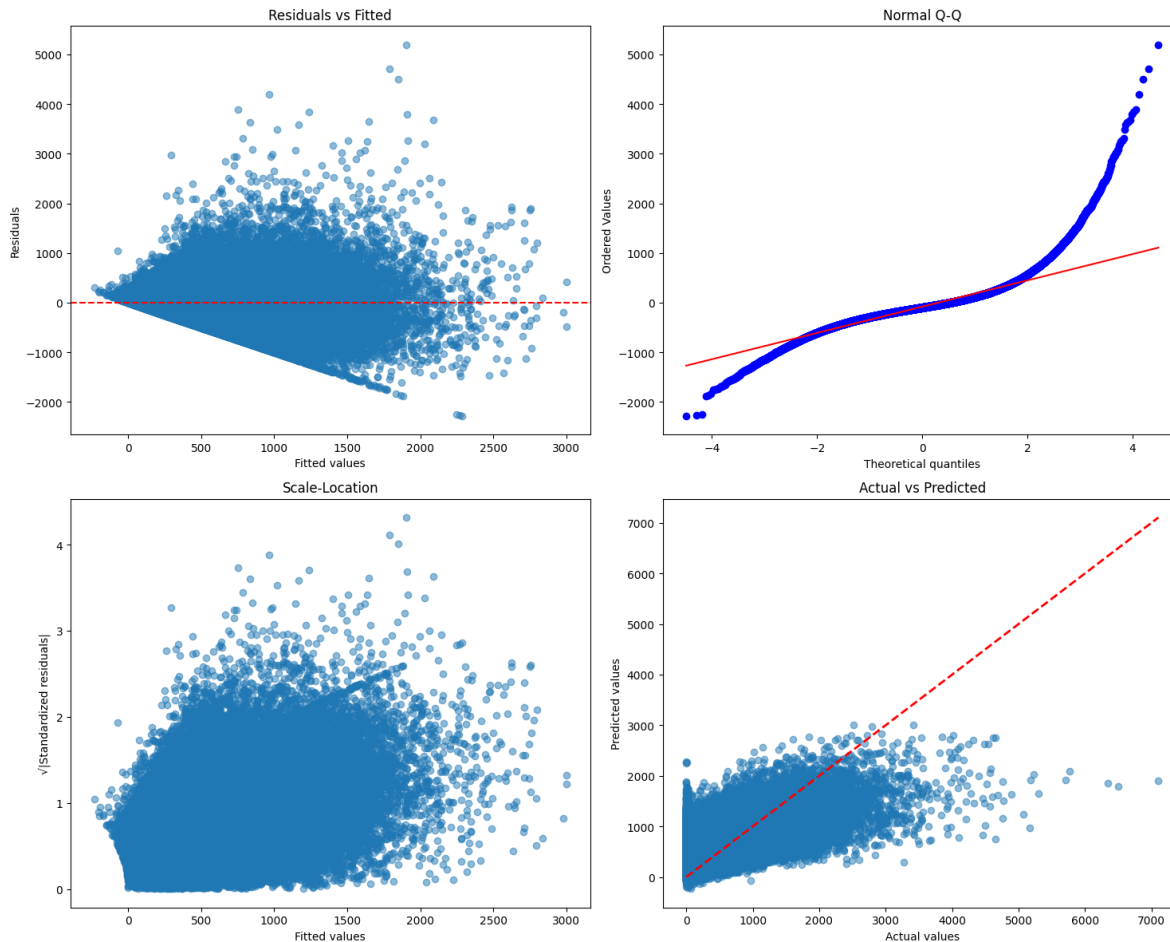
stats.probplot(residuals, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('Normal Q-Q')

# 3. Scale-Location (Spread-Location)
standardized_residuals = residuals / residuals.std()
axes[1, 0].scatter(y_pred, abs(standardized_residuals)**0.5, alpha=0.5)
axes[1, 0].set_xlabel('Fitted values')
axes[1, 0].set_ylabel('√|Standardized residuals|')
axes[1, 0].set_title('Scale-Location')

# 4. Residuals vs Leverage (using actual vs predicted instead)
axes[1, 1].scatter(y_true, y_pred, alpha=0.5)
axes[1, 1].plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'r--', lw=2)
axes[1, 1].set_xlabel('Actual values')
axes[1, 1].set_ylabel('Predicted values')
axes[1, 1].set_title('Actual vs Predicted')

plt.tight_layout()
mlflow.log_figure(fig, "regression_diagnostics.png")
plt.show()

```



```
mlflow.end_run()
```

View run beautiful-koi-920 at: <https://mlflow.analytics-dev.tapestry.com/#/experiments/81/r>

View experiment at: <https://mlflow.analytics-dev.tapestry.com/#/experiments/81>

```
# Create tenure buckets based on LOG_TENURE
tenure_bins = [float('-inf'), 0.25, 0.50, 0.75, 1.00, 1.25, float('inf')]
tenure_labels = ['<0.25', '0.25-0.50', '0.50-0.75', '0.75-1.00', '1.00-1.25', '>1.25']

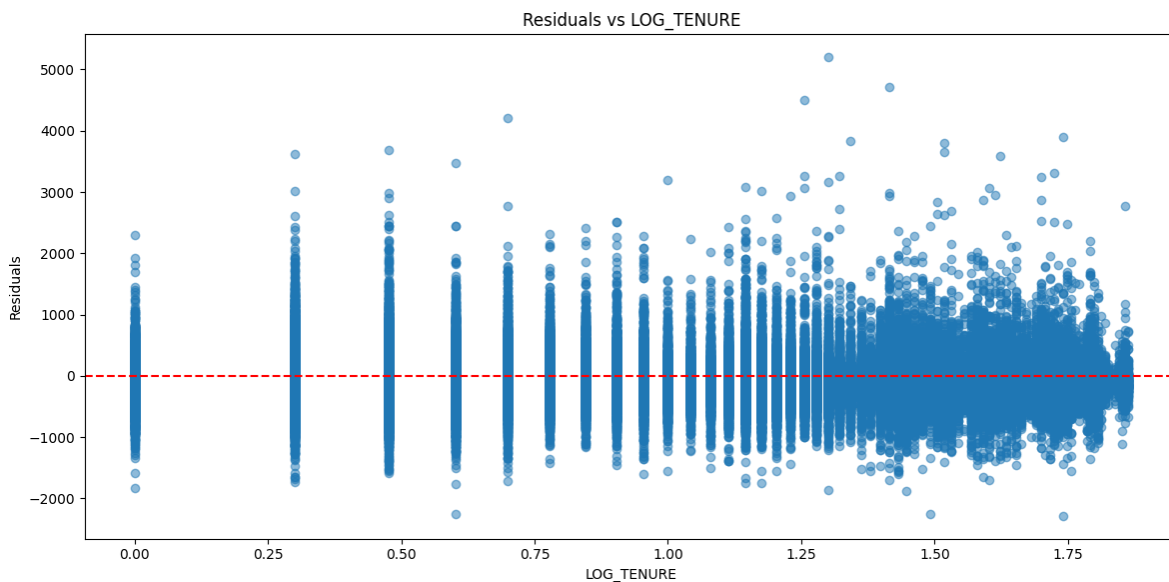
data['TENURE_BUCKET'] = pd.cut(data['LOG_TENURE'], bins=tenure_bins, labels=tenure_labels)

# Count individuals in each tenure range
tenure_counts = data['TENURE_BUCKET'].value_counts().sort_index()
print(tenure_counts)
print(f"\nTotal: {tenure_counts.sum()}")
```

```
TENURE_BUCKET
<0.25      9408
0.25-0.50  45997
0.50-0.75  21692
0.75-1.00  26003
1.00-1.25  26745
>1.25     63988
Name: count, dtype: int64
```

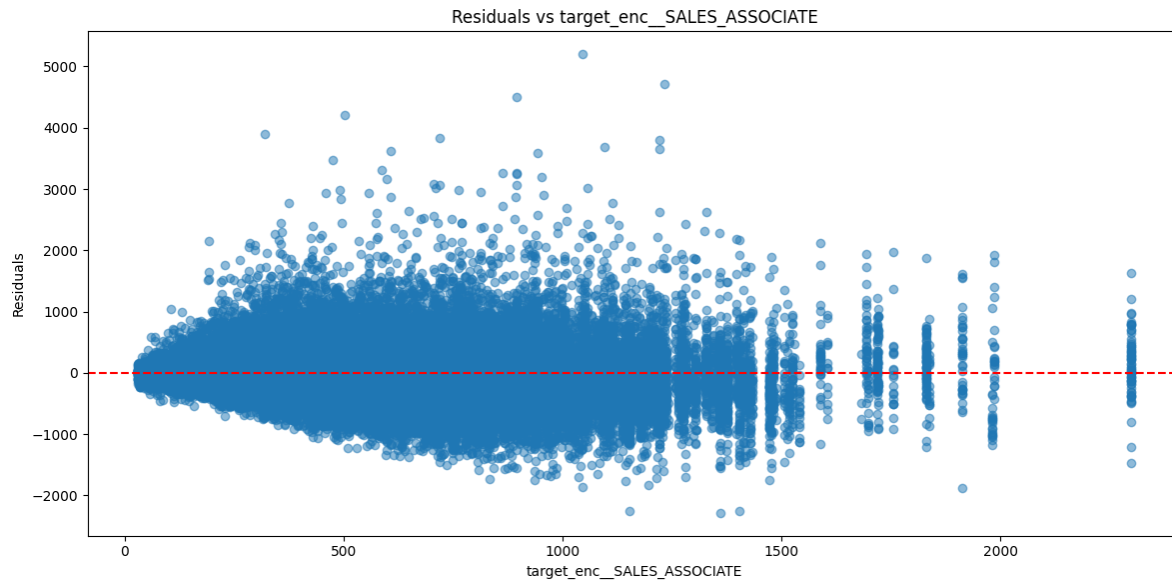
Total: 193833

```
plt.figure(figsize=(12, 6))
plt.scatter(data['LOG_TENURE'], residuals, alpha=0.5)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('LOG_TENURE')
plt.ylabel('Residuals')
plt.title('Residuals vs LOG_TENURE')
plt.tight_layout()
plt.show()
```

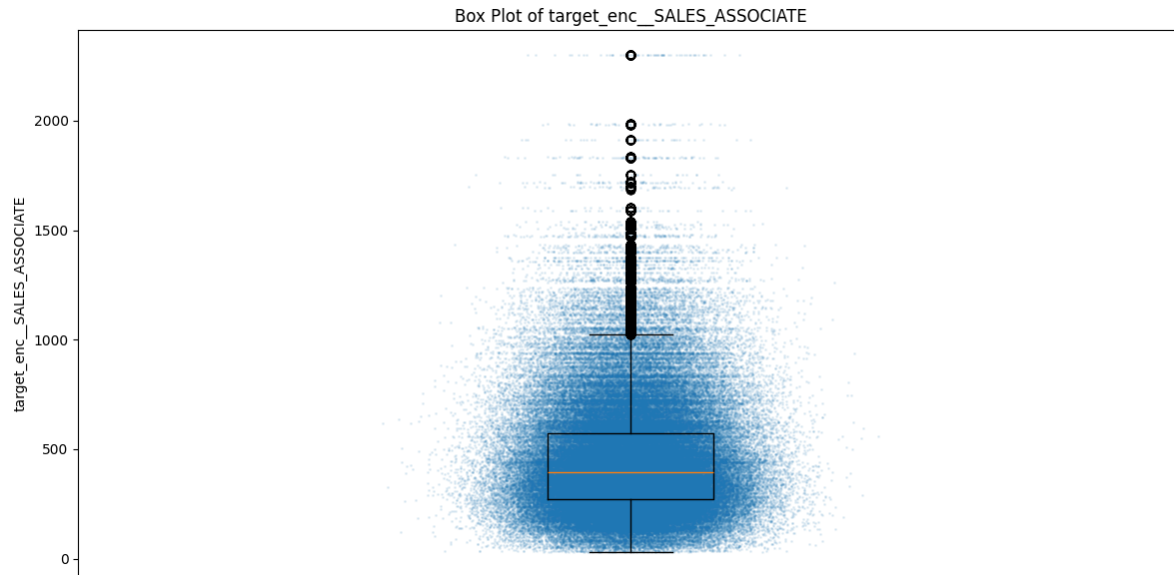


```
plt.figure(figsize=(12, 6))
plt.scatter(X_form['target_enc__SALES_ASSOCIATE'], residuals, alpha=0.5)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('target_enc__SALES_ASSOCIATE')
plt.ylabel('Residuals')
```

```
plt.title('Residuals vs target_enc__SALES_ASSOCIATE')
plt.tight_layout()
plt.show()
```

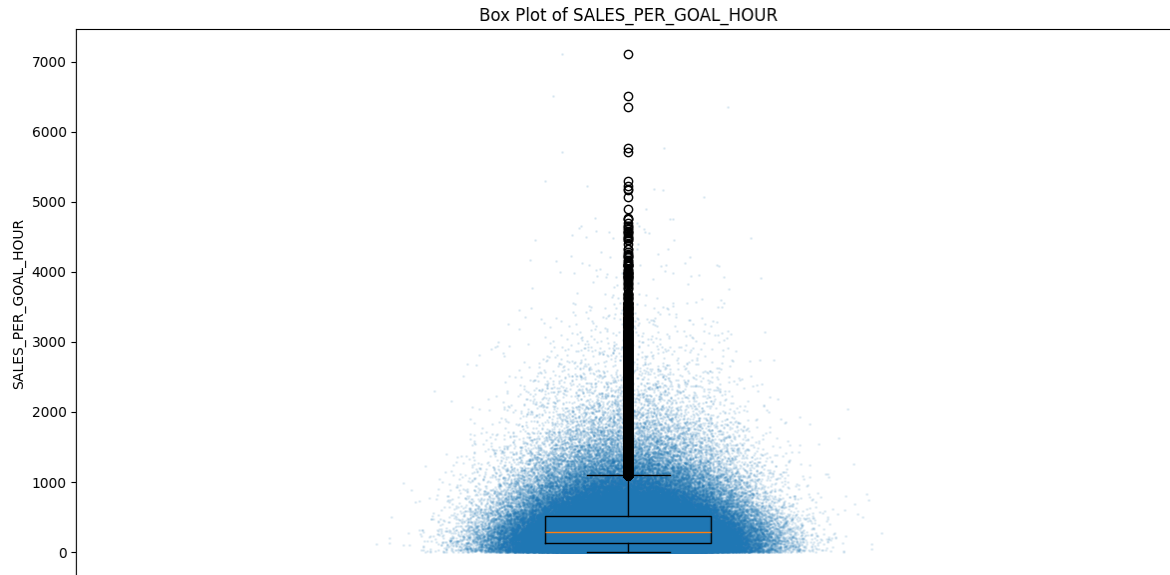


```
import numpy as np
plt.figure(figsize=(12, 6))
plt.boxplot(X_form['target_enc__SALES_ASSOCIATE'], vert=True)
plt.scatter(np.ones(len(X_form)) + np.random.normal(0, 0.05, len(X_form)),
            X_form['target_enc__SALES_ASSOCIATE'], alpha=0.1, s=1)
plt.ylabel('target_enc__SALES_ASSOCIATE')
plt.title('Box Plot of target_enc__SALES_ASSOCIATE')
plt.xticks([])
plt.tight_layout()
plt.show()
```



```
records = X_form[X_form['target_enc__SALES_ASSOCIATE'] > 2000]
```

```
import numpy as np
plt.figure(figsize=(12, 6))
plt.boxplot(data['SALES_PER_GOAL_HOUR'], vert=True)
plt.scatter(np.ones(len(data)) + np.random.normal(0, 0.05, len(data)),
            data['SALES_PER_GOAL_HOUR'], alpha=0.1, s=1)
plt.ylabel('SALES_PER_GOAL_HOUR')
plt.title('Box Plot of SALES_PER_GOAL_HOUR')
plt.xticks([])
plt.tight_layout()
plt.show()
```



```
# Calculate IQR for SALES_PER_GOAL_HOUR
q1 = data['SALES_PER_GOAL_HOUR'].quantile(0.25)
q3 = data['SALES_PER_GOAL_HOUR'].quantile(0.75)
iqr = q3 - q1
median = data['SALES_PER_GOAL_HOUR'].median()

print(f"SALES_PER_GOAL_HOUR IQR Analysis:")
print(f"Q1 (25th percentile): {q1:.2f}")
print(f"Median (50th percentile): {median:.2f}")
print(f"Q3 (75th percentile): {q3:.2f}")
print(f"IQR (Q3 - Q1): {iqr:.2f}")
print(f"\nOutlier bounds (using 1.5 * IQR):")
print(f"Lower bound: {q1 - 1.5 * iqr:.2f}")

print(f"Upper bound: {q3 + 1.5 * iqr:.2f}")
```

```
SALES_PER_GOAL_HOUR IQR Analysis:
Q1 (25th percentile): 133.02
Median (50th percentile): 283.38
Q3 (75th percentile): 520.46
IQR (Q3 - Q1): 387.44
```

```
Outlier bounds (using 1.5 * IQR):
Lower bound: -448.14
Upper bound: 1101.62
```